Compte rendu de réunion n°3 : TER BYZFL

Restructuration du code, Environnement GPU et Plan d'expérimentations

ABBES Samy & AREZKI Célia

Semaine du 18 Mai 2026

1 Objectifs de la réunion

Suite à nos premières séries de tests, cette réunion a permis de faire le point sur la qualité de notre base de code, les problèmes d'environnement rencontrés (matériels et logiciels) et de définir précisément le cadre de nos prochaines expérimentations.

2 Restructuration du code et Reproductibilité

2.1 Paramétrage via fichiers JSON

Il a été constaté une forte duplication de code dans nos scripts actuels, nous devions souvent réécrire ou copier-coller des fichiers entiers pour ne modifier que quelques lignes d'une expérience à l'autre. Pour pallier ce problème, il nous a été conseillé d'adopter une architecture basée sur des fichiers de configuration `.json`. Désormais, le script principal d'exécution ne contiendra que la logique globale et les variables de base (nombre de rounds, taille de batch, nombre de clients totaux et nombre de byzantins). Le reste des paramètres spécifiques sera injecté via le JSON :

- Choix de l'agrégateur.
- Type d'attaque byzantine à simuler.
- Distribution des données (IID ou Non-IID).
- Configurations spécifiques du serveur et des clients.

Ainsi, le lancement d'une nouvelle expérience se résumera à appeler le script principal en lui passant en argument le fichier de configuration souhaité.

2.2 Suivi des métriques et Validation par l'aléatoire

Afin de produire des analyses rigoureuses, nous devons systématiser la sauvegarde de l'évolution de l'*accuracy* et de la *loss* à intervalle de round régulier. Cela nous permettra de tracer des courbes d'apprentissage complètes plutôt que de se fier uniquement au résultat final. De plus, suite aux directives de notre encadrant, nous devons impérativement conserver le facteur aléatoire lors de nos simulations (ne pas fixer les *seeds* de NumPy et PyTorch). L'objectif est de vérifier que les conclusions et les règles théoriques que nous tirons de nos expériences restent valides et constantes dans tous les scénarios, indépendamment des aléas d'initialisation du modèle ou du bruit de sélection des données.

3 Environnement Technique et Résolution des problèmes

3.1 Justification de la version de PyTorch via UV

Une remarque a été faite concernant l'utilisation d'une version jugée « trop ancienne » de PyTorch (version 2.3). Après analyse approfondie du gestionnaire d'environnement `uv` (outil

que nous découvrons), il s'avère que ce choix est imposé par la bibliothèque `ByzFL` elle-même. `ByzFL` a été développée et testée spécifiquement avec PyTorch 2.2. Malgré nos tentatives de forcer des versions plus récentes dans le `requirements.txt`, `uv` résout automatiquement l'arbre des dépendances en téléchargeant la version 2.3, qu'il juge être la plus récente tout en restant strictement rétrocompatible avec les exigences internes de `ByzFL`. Nous conserverons donc cette version pour garantir la stabilité de nos scripts.

3.2 Accélération CUDA et transition vers Google Colab

La bibliothèque `ByzFL` propose deux modes d'exécution : CPU ou CUDA. Pensant que tous les ordinateurs portables étaient équipés d'une carte graphique NVIDIA exploitable, nous avions laissé `uv` gérer les dépendances par défaut. Suite à la demande d'exécuter des tests avec accélération matérielle, nous avons constaté que nos machines personnelles ne possédaient pas de véritable GPU dédié, mais uniquement des composants graphiques intégrés non compatibles avec CUDA. Pour remédier à ce blocage matériel, nous avons établi un protocole de migration de nos exécutions vers **Google Colab**. Toutes les étapes nécessaires pour configurer l'environnement distant et allouer une carte graphique performante (GPU NVIDIA T4) ont été documentées et regroupées dans le fichier `commandes_ggl_colab.txt`.

4 Plan d'expérimentation et Nouveaux jeux de données

4.1 Méthodologie de comparaison des agrégateurs

Pour évaluer efficacement la robustesse des défenses, le protocole expérimental consistera à **fixer une seule attaque par série de tests**. Face à cette attaque spécifique, nous ferons concourir différents agrégateurs. Cette méthode permettra d'isoler le comportement de chaque défense face à une menace donnée.

4.2 Introduction du jeu de données Fashion-MNIST

Afin de faire des expérience sur un problème plus complexe que la reconnaissance de chiffres manuscrits (MNIST), nous avons intégré le jeu de données **Fashion-MNIST**.

- **Description :** C'est un dataset de classification d'images d'articles de vêtements : T-shirt/Haut, Pantalon, Pull, Robe, Manteau, Sandale, Chemise, Basket, Sac, Bottine.
- **Caractéristiques :** Il est structuré exactement comme MNIST (60 000 images d'entraînement et 10 000 images de test, réparties en 10 classes). Les images sont en niveaux de gris et de dimension 28×28 pixels.
- **Intérêt :** Bien que la structure informatique soit identique (ne nécessitant que peu de changements dans le code), les motifs visuels sont beaucoup plus complexes à identifier que de simples chiffres. Il représente un défi plus réaliste pour évaluer la dégradation des performances lors d'une attaque byzantine.